

PORTADA

# **Análisis y Diseño de Sistemas II**

**Semana 15 — Código Limpio I: Nombres, Funciones, Comentarios**

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**El código se lee muchas más veces  
de las que se escribe**

TRANSICIÓN

## De la deuda técnica a la primera forma de pagarla

El martes se vio que la deuda técnica se paga como **fricción**: código cada vez más lento y arriesgado de cambiar. La forma más directa de reducir esa fricción no requiere una herramienta nueva ni una reunión — es escribir código que se explique a sí mismo.

Hoy se ve la primera mitad de **código limpio**, basada en el libro *Clean Code* (2008) de **Robert C. Martin** ("Uncle Bob"): nombres significativos, funciones pequeñas y el rol real de los comentarios. La segunda mitad —refactorización y code smells— se ve el miércoles.

CONCEPTO

## Por qué importa el código limpio

Robert C. Martin parte de una proporción concreta en la vida real de cualquier proyecto: por cada línea de código nueva que se **escribe**, un desarrollador va a **leer** código existente muchas más veces — al depurar, al agregar una funcionalidad relacionada, al hacer code review, al volver a un módulo meses después.

De ahí la idea central del libro: la **relación entre tiempo de lectura y tiempo de escritura** es tan desbalanceada que hace que escribir código fácil de leer sea, en la práctica, más importante que escribirlo rápido. Código limpio no es una cuestión de estética — es una inversión que se recupera cada vez que alguien (incluido uno mismo) vuelve a leerlo.

## CONCEPTO

# Nombres significativos (I)

El nombre de una variable, función o clase debería responder, sin necesidad de un comentario aparte, tres preguntas: **por qué existe, qué hace y cómo se usa**. Reglas centrales del capítulo de nombres en Clean Code:

- **Intención reveladora:** el nombre debe decir qué representa, no requerir que alguien lea el cuerpo del código para entenderlo.
- **Evitar desinformación:** no llamar `listaDeCuentas` a algo que en realidad es un mapa, ni usar nombres que se parezcan demasiado entre sí ( `datos` vs. `dato` ).
- **Hacer distinciones significativas:** evitar nombres como `datos1` , `datos2` o sufijos vacíos ( `ClienteInfo` cuando ya existe `Cliente` ) que no aportan información real.

## CONCEPTO

# Nombres significativos (II)

- **Pronunciables:** un nombre que no se puede decir en voz alta ( `genymdhms` para "generation year, month, day, hour, minute, second") hace imposible discutirlo en equipo.
- **Buscables:** preferir `DIAS_MAXIMO_RETRASO` a un número mágico como `7` suelto en el código — un nombre se puede buscar con "Find"; una constante numérica se confunde con cualquier otro `7` del archivo.
- **Evitar codificaciones tipo húngara:** prefijos como `strNombre` o `iContador` eran útiles cuando los editores no mostraban tipos — hoy son ruido que el lenguaje o el IDE ya resuelven.
- **Un concepto, una palabra:** no mezclar `obtener`, `traer` y `recuperar` para la misma operación en distintas clases del mismo proyecto.



## CONTRAEJEMPLO

# Antes y después: nombrar con intención

```
// Antes: nombres que no revelan intención
int d;           // días transcurridos
List<int[]> l = getList();
for (int[] x : l) {
    if (x[0] == 4) l2.add(x);
}

// Después: el nombre reemplaza al comentario
int diasTranscurridos;
List<Celda> celdasMarcadas = tablero.obtenerCeldas();
for (Celda celda : celdasMarcadas) {
    if (celda.getEstado() == ESTADO_MARCADA) {
        celdasCandidatas.add(celda);
    }
}
```

La segunda versión no necesita el comentario "días transcurridos" porque el nombre ya lo dice, y reemplaza `x[0] == 4` (un número mágico sin significado visible) por una constante con nombre.

## CONCEPTO

# Funciones pequeñas (I): hacer una sola cosa

La primera regla de Martin sobre funciones es contundente: **deben ser pequeñas**, y luego **más pequeñas todavía**. Una función larga casi siempre es una señal de que está haciendo más de una cosa — el mismo problema de baja cohesión que ya vieron en la semana 7, aplicado ahora al nivel de la función y no de la clase.

Una función **hace una sola cosa** cuando todas sus instrucciones están al mismo nivel de abstracción y contribuyen a un único propósito identificable. Una buena prueba práctica: si se puede extraer un bloque de la función y ponerle un nombre que no sea simplemente una repetición de su implementación, esa función estaba haciendo más de una cosa.

## CONCEPTO

# Funciones pequeñas (II): nivel de abstracción y argumentos

- **Un solo nivel de abstracción por función:** no mezclar una línea de alto nivel ("procesar pedido") con detalle de bajo nivel ("concatenar el string del encabezado HTTP") en la misma función.
- **Pocos argumentos:** el ideal es cero o uno; dos es aceptable; tres empieza a justificar revisión; cuatro o más casi siempre indica que esos argumentos deberían agruparse en un objeto propio.
- **Sin efectos secundarios ocultos:** una función llamada `validarPassword(password)` que además, sin decirlo en su nombre, inicia una sesión de usuario, engaña a quien la llama — hace algo que su nombre no anuncia.
- **Preferir excepciones a códigos de error:** devolver códigos de error obliga a quien llama a verificar el resultado inmediatamente, mezclando el flujo normal con el manejo de errores.



## CONTRA EJEMPLO

# Una función que hace demasiado

```
def procesar_pedido(pedido):  
    if pedido.items == []:  
        return False  
    total = 0  
    for item in pedido.items:  
        total += item.precio * item.cantidad  
    if pedido.cliente.tipo == "premium":  
        total = total * 0.9  
    conexion = conectar_base_datos()  
    conexion.execute("INSERT INTO pedidos ...")  
    enviar_email(pedido.cliente.email, "Pedido confirmado")  
    return True
```

Esta función **valida, calcula el total, aplica un descuento, persiste y notifica** — cinco responsabilidades distintas en cinco niveles de abstracción distintos. Extraer cada bloque en una función propia ( `calcularTotal` , `aplicarDescuento` , `guardarPedido` , `notificarCliente` ) deja `procesar_pedido` como una lista legible de pasos de alto nivel.

## CONCEPTO

# Comentarios: por qué la mayoría son un síntoma

Martin es deliberadamente provocador en este punto: "**cada comentario es un fracaso de nuestra parte para expresarnos en código**". Un comentario que explica *qué* hace un bloque de código casi siempre significa que ese código pudo haberse escrito de forma que no necesitara explicación — con mejores nombres, funciones más pequeñas.

El riesgo adicional de un comentario es que, a diferencia del código, **no se ejecuta**: nada obliga a mantenerlo sincronizado cuando el código cambia. Un comentario desactualizado es peor que no tener comentario, porque miente con autoridad.

## CONCEPTO

# Cuándo un comentario SÍ es apropiado

Clean Code no dice "nunca comenten" — dice que un buen comentario compensa nuestra incapacidad de expresar algo en el código mismo, y lista casos legítimos:

- **Advertencias de consecuencias:** ej. `// no ejecutar en paralelo: la conexión no es thread-safe` .
- **Intención legal:** encabezados de licencia o copyright requeridos contractualmente.
- **Explicación de intención:** por qué se eligió un algoritmo poco obvio en vez de uno más simple.
- **TODO :** marcar trabajo pendiente conocido — de hecho, es una de las formas más simples de dejar visible una deuda técnica deliberada, conectando con la sesión del martes.
- **Amplificación de importancia:** señalar que algo que parece trivial en realidad es crítico.

## CONTRAEJEMPLO

# Comentario redundante vs. comentario que aporta

```
// Redundante: repite lo que el código ya dice
// Incrementa i en 1
i = i + 1;

// Apropiado: advierte algo que el código no puede decir por sí solo
// CUIDADO: este orden importa – el descuento debe aplicarse
// ANTES de calcular el impuesto, o el cliente paga de más.
total = aplicarDescuento(subtotal);
total = aplicarImpuesto(total);
```

El primer comentario no agrega nada que `i = i + 1` no diga ya. El segundo explica una **intención no obvia** (el orden de las operaciones) que ningún nombre de variable podría comunicar por sí solo — ese es el tipo de comentario que vale la pena escribir.

## INTERACCIÓN

# ¿Qué está mal en cada fragmento?

En el chat, para cada fragmento indiquen si el problema es de **nombres**, de **funciones que hacen demasiado**, o de **comentarios innecesarios**:

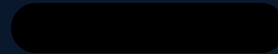
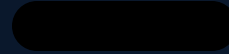
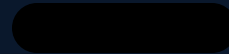
1. `def calc(a, b, c, d, e, f): ...` — una función con seis parámetros sueltos.
2. `// suma los dos números justo encima de total = precio + impuesto;`
3. Una función `guardarUsuario(usuario)` que, además de guardar, también envía un correo de bienvenida sin que su nombre lo mencione.

5 minutos · respondan en el chat

## SÍNTESIS

# Ideas clave de la sesión

1. El código se lee muchas más veces de las que se escribe — código limpio es una inversión, no estética.
2. Un buen nombre revela intención, evita desinformación, es pronunciable y buscable.
3. Una función debe ser pequeña, hacer una sola cosa, mantener un único nivel de abstracción y usar pocos argumentos.
4. La mayoría de comentarios son síntoma de código que no se explica solo; los buenos comentarios explican intención, advierten o marcan `TODO`.
5. Nombres y funciones limpias son la primera forma concreta de reducir el interés de la deuda técnica vista el martes.



CIERRE

## Antes de la próxima sesión

Revisen un fragmento de su propio proyecto y anoten, informalmente, si algún nombre o función viola alguna de las reglas vistas hoy — no hace falta corregirlo todavía. El viernes se completa código limpio con **refactorización y code smells**: el catálogo de Martin Fowler y las técnicas concretas para pasar del código de hoy al código limpio.

